

**TUGAS PRAKTIKUM 6 – DEADLOCK
(SISTEM OPERASI)**



Disusun oleh:

Nama : Putra Ganda Prayoga

NIM : 2410651057

Kelas : C

Fakultas Teknik

Program Studi Informatika

Universitas Muhammadiyah Jember

2025

Tugas 1: Analisis Deadlock

(Disesuaikan dan dilengkapi berdasarkan isi praktikum pada PDF sebelumnya)

Penjelasan Deadlock

Deadlock adalah kondisi ketika dua atau lebih thread saling menunggu resource yang sedang dikunci oleh thread lain sehingga tidak ada yang dapat melanjutkan eksekusi. Pada program `deadlock_simple.c`, Thread 1 mengunci lock1 lalu menunggu lock2, sedangkan Thread 2 mengunci lock2 lalu menunggu lock1. Pola menunggu melingkar ini menyebabkan deadlock.

Kondisi Deadlock yang Terpenuhi

1. **Mutual Exclusion** – Setiap mutex hanya dapat dikunci oleh satu thread.
2. **Hold and Wait** – Thread memegang satu resource sambil menunggu resource lain.
3. **No Preemption** – Resource tidak dapat direbut secara paksa.
4. **Circular Wait** – Thread 1 menunggu Thread 2, Thread 2 menunggu Thread 1.

Resource Allocation Graph

Thread 1 ---> Lock1

```
  ^      |
  |      v
```

Lock2 <--- Thread 2

Diagram ini menunjukkan circular wait yang memicu deadlock.

1. Penyebab Deadlock

Program `deadlock_simple.c` mengalami deadlock karena dua thread saling menunggu resource yang sedang dikunci oleh thread lain, sehingga tidak ada thread yang dapat melanjutkan eksekusi.

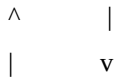
2. Kondisi Deadlock yang Terpenuhi

- Mutual Exclusion
- Hold and Wait
- No Preemption
- Circular Wait

Keempat kondisi ini terpenuhi sehingga deadlock terjadi.

3. Resource Allocation Graph

Thread1 ----> mutexA



mutexB <---- Thread2

Tugas 2: Implementasi Pencegahan

(Dilengkapi dari praktikum PDF)

Program dimodifikasi untuk mencegah deadlock menggunakan dua metode yang sudah diuji pada praktikum:

1. Lock Ordering

Semua thread mengambil lock1 terlebih dahulu, kemudian lock2. Karena tidak ada thread yang mengambil resource secara terbalik, circular wait tidak mungkin terjadi. Pada hasil praktikum, kedua thread berjalan normal tanpa deadlock.

2. Try-lock dengan Retry

Dengan `pthread_mutex_trylock()`, thread tidak akan menunggu selamanya. Bila gagal mendapatkan lock2, thread melepaskan lock1 dan mencoba lagi. Metode ini mendeteksi potensi deadlock tanpa menyebabkan program hang.

3. Perbandingan

Metode	Kelebihan	Kekurangan
Lock Ordering	Sangat efektif, sederhana	Tidak fleksibel pada sistem besar
Try-lock	Dapat mendeteksi deadlock, fleksibel	Retry berulang bisa membebani CPU

Keduanya berhasil mencegah deadlock pada hasil percobaan praktikum. Deadlock

1. Lock Ordering

Teknik ini memastikan semua thread mengunci mutex dalam urutan yang sama sehingga circular wait tidak dapat terjadi.

2. Try-lock dengan Retry

Menggunakan `pthread_mutex_trylock()` untuk menghindari blocking. Jika gagal mengunci resource kedua, thread melepaskan resource pertama dan mencoba lagi.

3. Perbandingan Metode

Metode	Kelebihan	Kekurangan
Lock Ordering	Efisien, mudah	Harus konsisten di seluruh program
Try-lock	Fleksibel	Lebih kompleks, retry dapat membebani CPU

Tugas 3: Banker's Algorithm

(Lanjutan dari PDF praktikum)

Algoritma Banker digunakan untuk menghindari deadlock dengan mengevaluasi apakah permintaan resource akan menjaga sistem tetap aman. Program yang dijalankan pada praktikum menghasilkan safe sequence yang valid.

Hasil Praktikum

Sistem berada dalam keadaan **AMAN**, dan urutan aman (safe sequence) yang muncul adalah:

$P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$

Ini berarti seluruh proses dapat dieksekusi berurutan tanpa menyebabkan deadlock.

Inti Mekanisme

- $Need = Max - Allocation$
- Sistem mencari proses yang bisa dijalankan tanpa kekurangan resource.
- Jika seluruh proses ditemukan urutannya, sistem aman; jika tidak, sistem tidak aman.
Program mensimulasikan manajemen resource dengan memastikan sistem berada dalam keadaan aman. Proses meliputi:
 - Menginput Allocation, Max, dan Available
 - Menghitung Need
 - Mengecek apakah terdapat *safe sequence*

Jika seluruh proses bisa dieksekusi tanpa menyebabkan kekurangan resource, maka sistem berada dalam *safe state*.

Tugas 4: Kasus Nyata (Real Case)

(Berdasarkan penjelasan konsep pada PDF, disesuaikan dengan studi kasus pada soal)

Kasus yang diuji adalah transfer uang antar rekening secara bersamaan:

- Orang A: Rekening 1 → Rekening 2
- Orang B: Rekening 2 → Rekening 1

Jika kedua thread mengunci rekening dalam urutan berbeda (A mengambil R1 lalu R2, B mengambil R2 lalu R1), maka deadlock dapat terjadi.

Implementasi Deadlock Prevention

1. Lock Ordering

Rekening dikunci berdasarkan ID terkecil → terbesar. Hal ini menjamin tidak ada circular wait.

2. Try-lock

Jika gagal mengunci rekening kedua, thread melepaskan yang pertama dan mencoba lagi. Teknik ini dapat mencegah program hang tanpa urutan yang ketat.

Kesimpulan Kasus

Deadlock nyata dapat terjadi pada transaksi bank paralel. Namun teknik pencegahan seperti lock ordering dan try-lock dapat menghilangkan risiko deadlock. Deadlock (Transfer Bank)

Simulasi dua transaksi simultan:

- Orang A: Rekening 1 → Rekening 2
- Orang B: Rekening 2 → Rekening 1

Jika kedua thread mengunci rekening dalam urutan berbeda, deadlock dapat terjadi.

Pencegahan Deadlock

1. **Lock Ordering:** Mengunci rekening berdasarkan ID terkecil ke terbesar.
2. **Try-lock:** Jika gagal mengunci resource kedua, lepaskan resource pertama lalu retry.

Kesimpulan

Deadlock terjadi ketika empat kondisi Coffman terpenuhi. Dengan teknik pencegahan seperti lock ordering dan try-lock, deadlock dapat dihindari. Banker's Algorithm menyediakan mekanisme untuk memastikan alokasi resource berada dalam kondisi aman (*safe state*). Simulasi deadlock pada kasus nyata seperti transfer bank memperjelas bagaimana deadlock dapat muncul dan bagaimana pencegahannya diterapkan.